A COMPLETE PYTHON PROJECT

YOUR OWN AI

Python Desktop Edition

Vector search. Your documents. Optional local AI.

Professional project presentation and practical run guide

3
SEARCH ALGORITHMS

2
DOCUMENT MODES

0
CORE PIP PACKAGES

Source code, setup, architecture, usage, validation and future scope

PURPOSE

Find useful information. Understand the mechanism.

A practical document assistant and an educational vector-search laboratory.

The problem

Useful knowledge is spread across notes and files. A complex toolchain can make a first AI project difficult to run.

The approach

One desktop app combines transparent algorithms, persistent documents and an offline retrieval path.

The outcome

Run a useful demonstration first. Add local model inference when your computer and project need it.

Designed for learning, personal knowledge exploration and controlled prototypes.

PROJECT EVOLUTION

The original idea, rebuilt for Python

Based on the supplied Your-OWN-AI-main.zip project.

Area	Supplied project	Python edition
Application	C++ engine + browser interface	Python desktop application
Setup	C++ compiler and local AI setup	Python + Tk; optional models later
Vector search	HNSW, KD-Tree, brute force	Readable Python implementations of all three
Documents	Ollama embedding / RAG workflow	Offline extracts + optional Ollama RAG
Persistence / delivery	Original source archive	SQLite, backups, launchers, tests and documentation

Project-specific comparison: supplied source archive and delivered Python source.

DELIVERED CAPABILITIES

A complete, understandable workflow

Two kinds of document answers are clearly labelled in the interface.

Explore vectors

Search synthetic 16D vectors; select a metric; inspect PCA; compare algorithms with measured timings.

Manage documents

Add or import notes, preserve chunks in SQLite, delete documents and export or merge JSON backups.

Find source evidence

Offline TF-IDF returns labelled extracts. Find passages and export an answer for later review.

Add local generation

With Ollama installed, create embeddings and generate answers using retrieved context. Models are separate downloads.

Python throughout the application

No C++ compilation. No hosted AI account needed for core functionality.

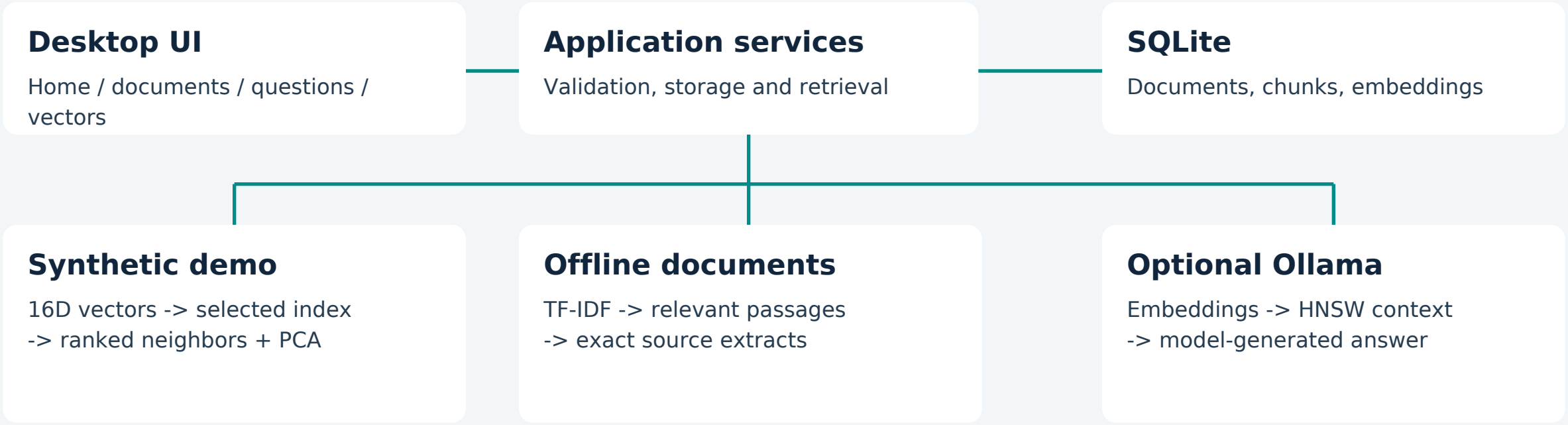
Technology	Use	Required?
Python 3.10+	Application source; 3.12 recommended	Yes
tkinter / ttk	Desktop pages, tables, dialogs and canvas	GUI; Tk support must be installed
sqlite3 + standard modules	Persistence, algorithms, HTTP, files, tests	Included with normal Python setup
pypdf	Selectable-text PDF import	Optional extra
Ollama + two models	Neural embeddings and generated answers	Optional separate installation
ReportLab	Regenerate this presentation PDF	Documentation only

References: Python Tkinter / sqlite3 docs; pypdf; Ollama API and model catalog. Full links on page 24.

ARCHITECTURE

One application, three processing paths

The desktop and local API share core services; durable content lives in SQLite.



The REST API is optional and loopback-only. Its document routes use offline retrieval.

Three implementations, different trade-offs

The exact baseline makes approximate results assessable.

Algorithm	How it searches	What to explain
Brute force	Compute every vector distance and rank results	Exact. O(N x d) distance work, plus result ranking.
KD-Tree	Split by coordinates and prune safe branches	Exact. High dimensions can weaken pruning; no universal logarithmic bound.
HNSW	Navigate a layered neighbor graph	Approximate. Recall depends on data, connectivity and search effort.

The delivered indexes are educational Python implementations. Small datasets may favor a direct scan.

Concept references: Malkov & Yashunin HNSW paper; SciPy KDTree documentation (not a project dependency).

Search in full dimensions. Plot a 2D projection.

The demo uses synthetic teaching vectors, not a neural text model.

Cosine

Compares direction.

distance = 1 - cosine similarity

Lower distance is better. Zero vectors require explicit handling.

Euclidean

Straight-line distance.

sqrt(sum of squared coordinate differences)

Feature scale matters.

Manhattan

Coordinate-wise distance.

sum of absolute coordinate differences

A different geometry can change ranking.

PCA keeps two directions of variation. It loses information, so screen distance is not the search result.

INTERFACE GUIDE

Five pages, clear actions

Navigation schematic based on the implemented UI; this is not an application screenshot.

Page	Primary actions	Use it to...
Home	Open the workspace; load sample documents	Start a demonstration and see status
Documents	Add / Import file / Export backup / Import backup	Manage your saved source material
Ask documents	Ask question / Find passages / Export answer	Find and inspect source evidence
Vector lab	Search vectors / Compare algorithms / Manage vectors	Explore algorithms and distances
Settings	Check connection / Save settings	Configure optional local models

The GUI needs a graphical desktop. Visual launch validation was unavailable in the build environment.

Use the included desktop checklist on your computer. Core and API validation are reported separately.

EXECUTED OFFLINE EXAMPLE

An answer you can trace to your notes

Actual core execution against bundled sample documents; no model inference was used.

Question

What is RAG?

What the result means

These are existing source sentences. They may include limitations as well as a definition.

Open the complete retrieved passages to understand the surrounding context.

Returned source extracts

Offline mode - matching source excerpts (not a generated answer):

[1] RAG does not automatically guarantee accuracy or eliminate hallucinations.

[2] RAG means Retrieval-Augmented Generation.

Open the cited passages to check the context. Keyword matches may not fully answer the question.

The example JSON is included as docs/offline_example.json. Source labels refer to the provided documents.

DOCUMENT RETRIEVAL

From your files to source-labelled extracts

Paste text, import UTF-8 TXT/Markdown, or optionally import a text-based PDF.

1. Add content

Documents > Add document or Import file.

Chunks contain up to 180 words with 35-word overlap.

2. Rank passages

TF-IDF weights terms and scores lexical overlap with the question.

This mode needs no downloaded AI model.

3. Review evidence

The output contains relevant text from your notes and source labels.

Unrelated questions may have no useful match.

Limits: 10 MB per input file; 200,000 text characters per document. Scanned PDFs require external OCR.

Concept reference: Stanford Introduction to Information Retrieval. PDF limits: pypdf extraction documentation.

OPTIONAL LOCAL AI

Add embeddings and generated answers with Ollama

Both models are separate downloads. Core offline use remains available.

```
ollama pull nomic-embed-text
ollama pull llama3.2:1b
ollama list
```

Connect

Start Ollama if needed. Open Settings, keep the local base URL, use Check connection, then Save settings.

Ask in AI mode

Choose Ollama (local AI). The app embeds documents, retrieves context and requests a generated answer. Review the sources.

Official sources: docs.ollama.com/api/embed, /api/generate, /api/tags and model catalog links on page 24.

INSTALLATION

Run the project on your computer

Recommended starting point: Python 3.12 with Tcl/Tk support; minimum Python 3.10.

Windows

1. Install Python and include Tcl/Tk.
2. Extract the entire ZIP.
3. Double-click START_WINDOWS.bat.

Terminal alternative:
py -3 app.py

Use python app.py if python is your installed command.

macOS / Linux

1. Install Python with matching Tk support.
2. Extract the entire ZIP.
3. Run python3 app.py.

Launcher alternative:
bash start.sh

On Debian/Ubuntu, missing Tk is commonly resolved with python3-tk.

Verify Tk first: python -m tkinter | A GUI session is required for the desktop window.

Python reference: docs.python.org/3/library/tkinter.html. Interpreter command names vary by platform.

Start small, then add only what you need

Commands run from the extracted project folder.

Goal	Command
Desktop application	python app.py
Offline diagnostic	python app.py --self-check
Automated tests	python -m unittest discover -s tests -v
A separate data folder	python app.py --data-dir ./my-data
Optional PDF import	python -m pip install -r requirements-pdf.txt
Optional local REST API	python app.py --api --port 8765

Replace python with python3 or py -3 when that is the command for your installed interpreter.

STORAGE AND SOURCE CODE

Persistent documents, readable modules

Default data folder: .your_own_ai under your home directory.

Durable data

knowledge.sqlite3 stores documents, chunks and model-specific cached embeddings.

settings.json stores preferences.

Backup behavior

Export backup writes document JSON. Import backup merges duplicates and rebuilds chunks.

Model files and embedding caches are not exported.

Module	Responsibility
gui.py / app.py	Interface and launch modes
vectors.py / knowledge.py	Search algorithms; document persistence and retrieval
ollama_client.py / api.py	Optional model requests; local JSON endpoints

Vector lab edits are session-only. Database relationships and complete module map are in PROJECT_REPORT.md.

DEVELOPER INTERFACE

A loopback API for scripts and demonstrations

Desktop use does not require this server.

Endpoint group	Purpose
GET /health	Health check without a token
GET /api/stats; GET /api/documents	Statistics and document listing
POST /api/documents; DELETE /api/documents/<id>	Add/delete saved documents
POST /api/search; POST /api/ask	Offline passage search and sourced extracts
/api/vectors; /api/vector-search; /api/benchmark	Session vector CRUD, search and algorithm comparison

The server prints a bearer token. Send Authorization: Bearer <token> on /api/ routes. GUI Ollama mode is separate.

Exact methods, JSON fields and limits are documented in PROJECT_REPORT.md. The API is not a public hosting service.

BENCHMARKING

Measure latency and recall together

No fixed speed claims: actual results depend on the dataset and computer.

Build time

Creating the index costs work before queries begin.

Keep build cost separate from query time when comparing methods.

Query time

A high-resolution timer measures repeated searches.

Tiny datasets can be dominated by Python overhead and background load.

Recall@k

Compare the returned IDs with the exact top-k reference.

4 of 5 recovered neighbors gives recall@5 = 0.8.

Recall measures neighbor agreement. It does not measure whether an AI answer is correct.

VALIDATION

What was checked before delivery

The evidence describes this release and its tested environment.

- 61 automated tests passed on Linux x86_64 with Python 3.12.14: vectors 10, actual HTTP API 9, documents/Ollama 35 and GUI controllers 7.
- App and shell-launcher self-checks passed. The standalone offline demonstration executed successfully.
- Optional PDF import: pypdf 6.10.0 extracted text from the 24-page presentation and returned matching passages.
- Python 3.10 syntax parsing passed. GUI controllers used widget stand-ins and real SQLite/worker logic; local-model requests used mocks.

Not exercised: real desktop rendering, Windows/macOS execution, Python 3.10 runtime and real Ollama model inference. Follow the included desktop checklist on your computer.

Detailed test commands and evidence are included in the package validation documentation.

Useful locally, with clear limits

This is an educational implementation and personal desktop prototype.

Retrieval limits

TF-IDF may miss synonyms. HNSW can miss neighbors. PDF text order can be imperfect. PCA is a lossy view.

Generation limits

Retrieved context can be incomplete. A language model can still add unsupported statements. Review sources.

Data handling

The database and exports are local and unencrypted. Use trusted documents and protect backups on your computer.

Deployment limits

A local API token is not a multi-user permissions system. Public deployment and production scale need further engineering.

A practical foundation for the next version

The items below are proposals, not delivered capabilities.

Next improvement	Why it could matter	How to evaluate
Hybrid retrieval + reranking	Improve relevant passage ranking	Labelled query set and retrieval recall
OCR / DOCX / page highlighting	Make more document formats useful	Import accuracy and source navigation
Optimized ANN + batch embeddings	Handle larger collections	Memory, build time and query latency
Multilingual evaluation	Support selected language needs	Native-language test questions
Installer + shared-use safeguards	Reduce setup effort; manage access	Install success and permissions tests

Potential impact: faster note lookup and clearer AI education. No business productivity gain is claimed as measured.

PRESENTATION DEMO

A five-minute live walkthrough

Use the included notes; optional local models should be installed before presenting.

Time	Action	What to explain
0:00-0:40	Show Home	Purpose and offline-first setup
0:40-1:30	Open Vector lab and search	Synthetic vectors; full-space distance; PCA
1:30-2:10	Compare algorithms	Exact reference and approximate recall
2:10-3:20	Ask documents: What is RAG?	Source extracts and persistent knowledge
3:20-4:10	Show Settings / optional Ollama	Embeddings versus generation
4:10-5:00	Close with testing and future scope	Honest limits and next experiment

QUESTIONS AND ANSWERS

Be ready to explain the design

Short answers for a project review or viva.

Is this a trained AI model?

No. The project implements search and retrieval. Optional pretrained models provide embeddings and generation.

Why Python and SQLite?

Python makes the logic readable. SQLite saves local documents without a database server. No C++ compilation is needed.

Why can brute force be faster?

On small datasets its simple scan can cost less than graph or tree overhead. Measure before choosing.

Do sources guarantee a correct answer?

No. Sources make evidence inspectable. The original text and any generated claim still need review.

KEY TERMS

The vocabulary behind the project

Keep representation, retrieval and generation separate.

Term	Meaning in this project
Embedding	A numeric vector produced by a model from text
Chunk	A passage used as a retrieval unit
TF-IDF	A term-weighting method for lexical relevance
HNSW / KD-Tree	Graph-based approximate search / tree-based exact spatial search
PCA	A 2D visualization made from higher-dimensional vectors
RAG	Retrieve context, then ask a model to generate a response

PRIMARY REFERENCES

Technical sources and project documentation

Official documentation and original research; full source list included in REFERENCES.md.

Python desktop and storage

https://docs.python.org/3/library/tkinter.html

https://docs.python.org/3/library/sqlite3.html

Ollama API and local models

https://docs.ollama.com/api/embed

https://docs.ollama.com/api/generate

Model catalog

https://ollama.com/library/nomic-embed-text

https://ollama.com/library/llama3.2:1b

HNSW research / KD-Tree reference

https://arxiv.org/abs/1603.09320

https://docs.scipy.org/doc/scipy/reference/generated/scipy.spatial.KDTree.html

TF-IDF / PDF extraction

https://nlp.stanford.edu/IR-book/html/htmledition/tf-idf-weighting-1.html

https://pypdf.readthedocs.io/en/stable/user/extract-text.html

Companion files: README.md for running the app; PROJECT_REPORT.md for full design and usage details.

Sources checked 9 September 2026. External documentation does not certify this implementation.